**ACTIVITAT AVALUABLE AC1**

**Mòdul:** Desplegament d'aplicacions web

**Professor:** Marc Callejón

**Data límit d'entrega:** 05/04/2026 23:59

**Mètode d'entrega:** Per mitjà del Clickedu de l'assignatura. Les activitats entregades més enllà de la data límit només podran obtenir una nota de 5.

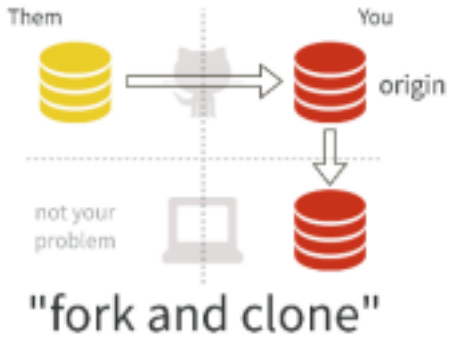
Es valorará la presentació.

**En esta práctica exploraremos algunos aspectos importantes del <u>despliegue *online* de un proyecto</u>. Para ello, utilizaremos el proyecto en NodeJS que encontraréis en el repositorio de más abajo**

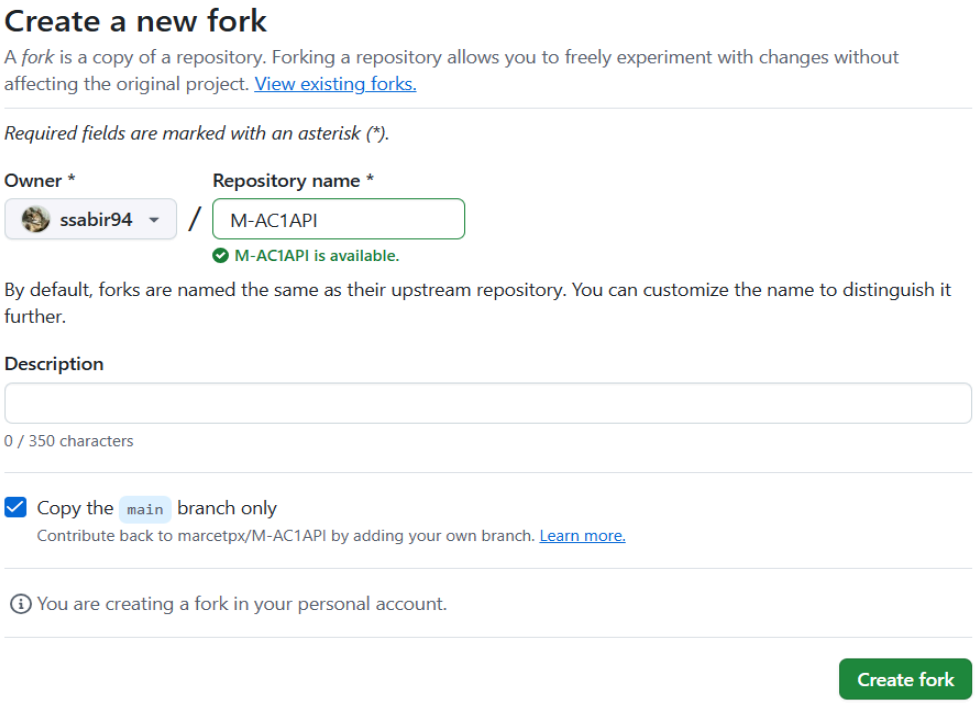
**Es importante, para el correcto desempeño de la práctica, documentar todos los pasos del proceso.**

## Despliegue del proyecto en local

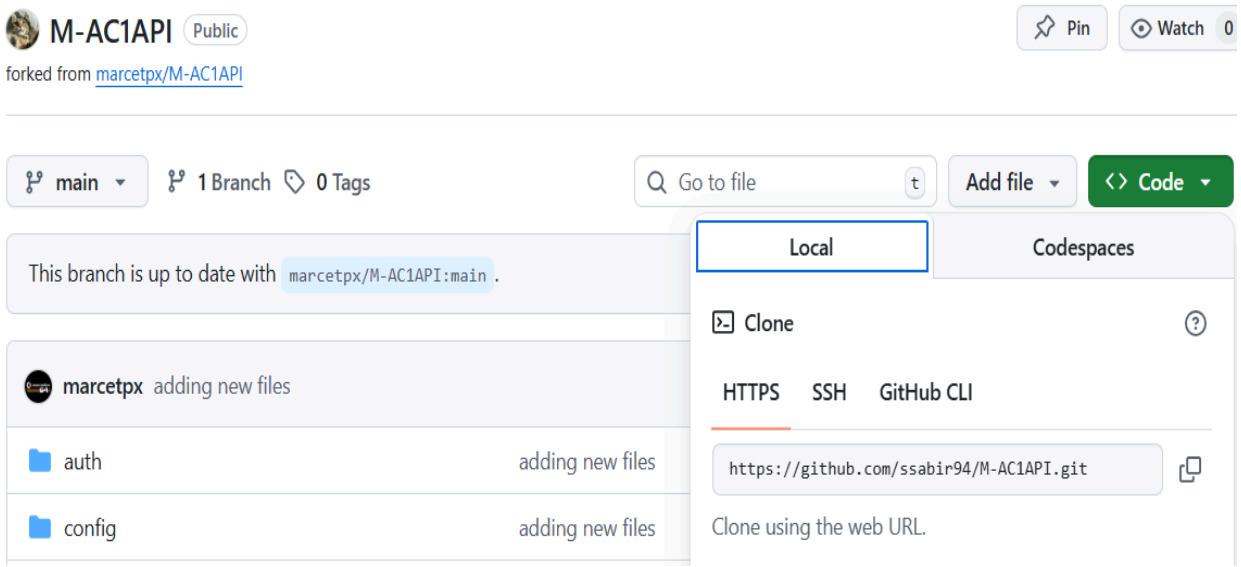
**Tarea 1. Realiza un *fork* del siguiente repositorio a tu GitHub, y clona este a tu máquina local. https://github.com/marcetpx/M-AC1API/tree/main**



## Primero hacemos un Fork :)



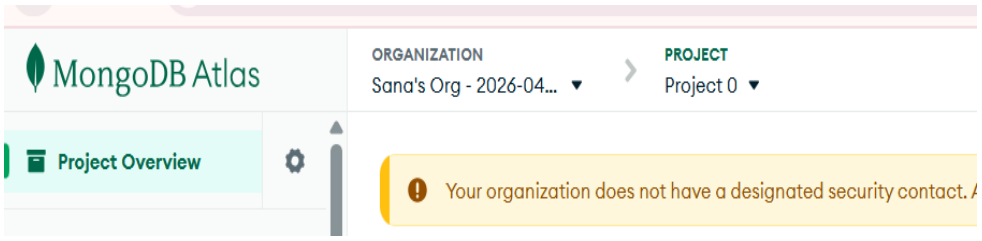

## Ahora clonamos:

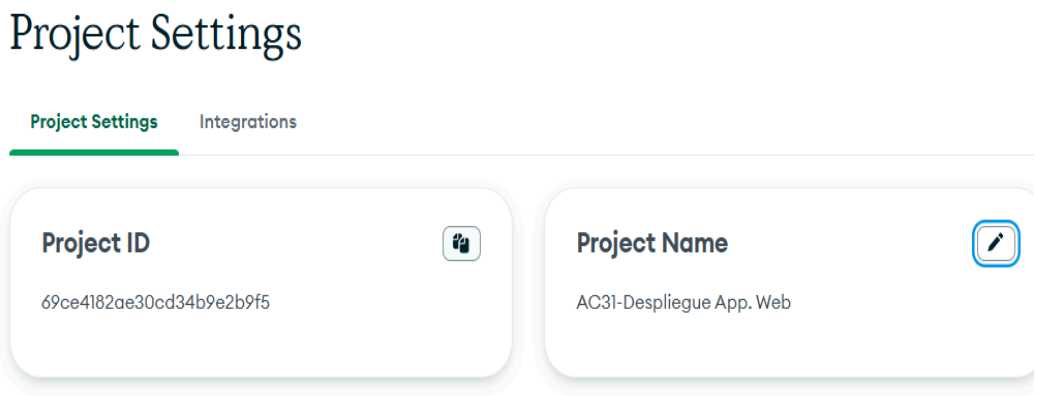


**El proyecto que encontrarás en el repositorio necesita de dos pasos previos antes de poder ser ejecutado en local.**

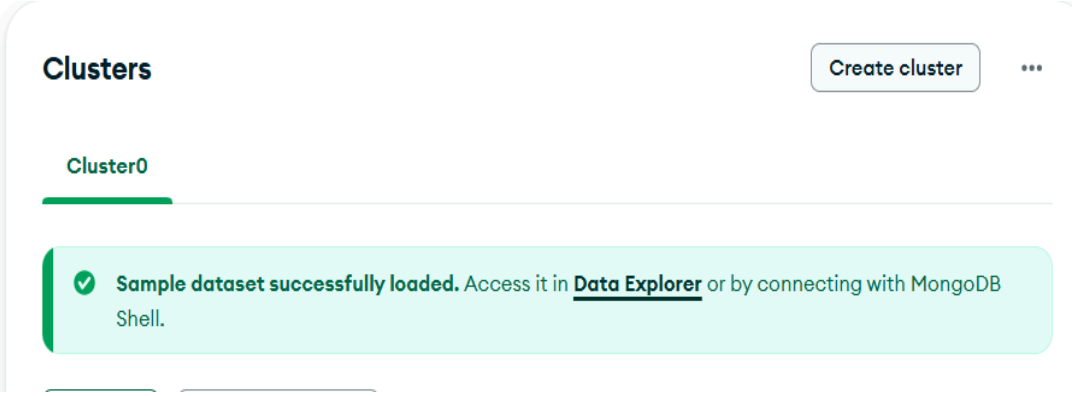
**a. Dado que utiliza una conexión a MongoDB, tendrás que configurar una cuenta, un clúster y una base de datos en el *cloud* Atlas de MongoDB (tarea 2).**

**Config. Cuenta:**

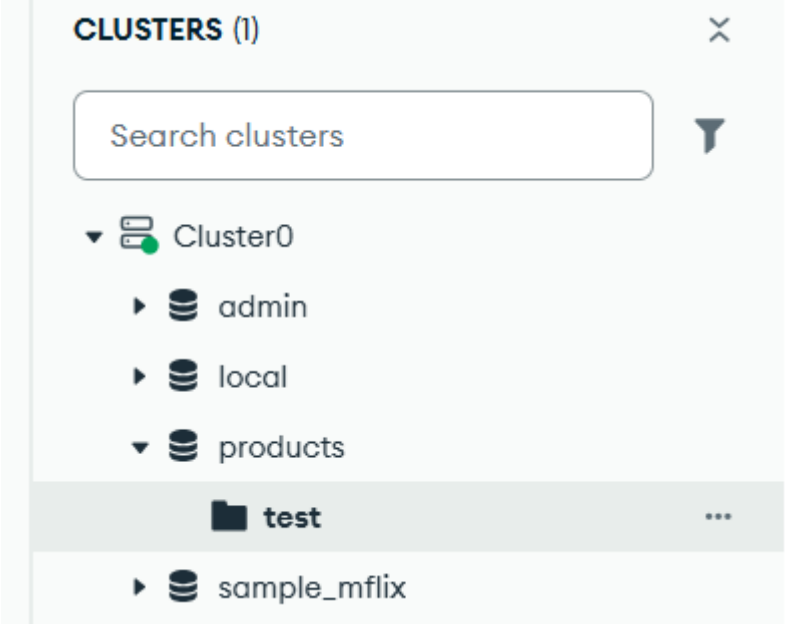



**Clúster:**



**Base de datos:**




**b. Dado que no se incluyen las dependencias (módulos de *Node*) en el repositorio, se deberán instalar manualmente mediante *npm* (tarea 3).**

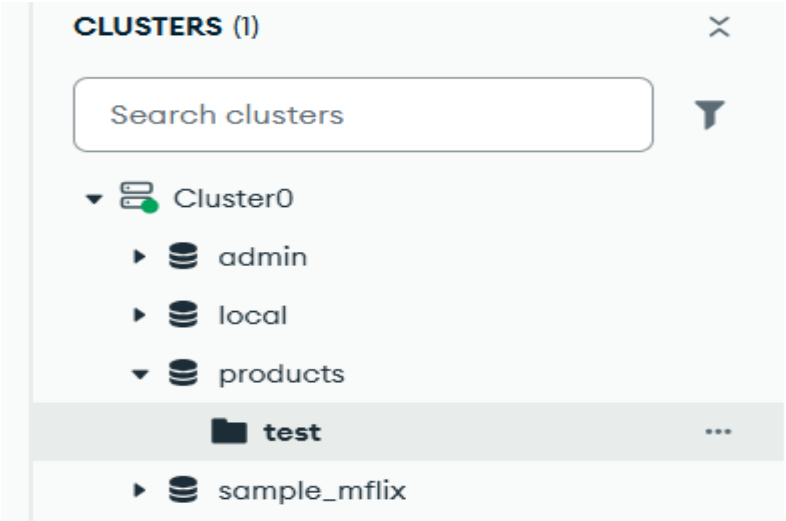
```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm install

added 207 packages, and audited 208 packages in 29s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**Tarea 2. Sigue las instrucciones que encontrarás en el enlace siguiente para crear la base de datos en Atlas.**

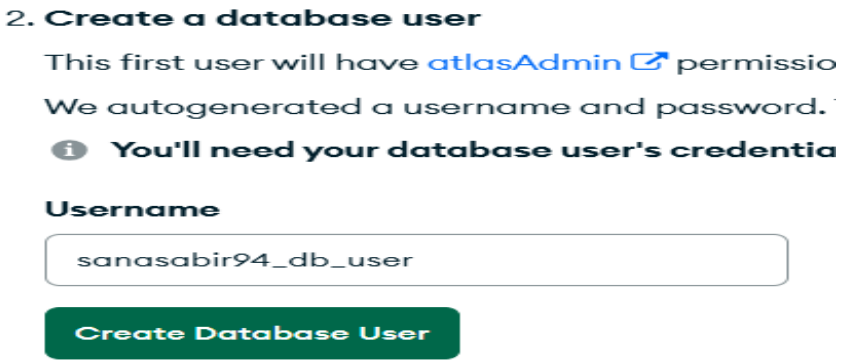
https://www.mongodb.com/docs/atlas/getting-started/

**Ten en cuenta que, para que nuestra aplicación pueda utilizar el clúster debes asegurarte de hacer las siguientes operaciones:**

**a. Crear un base de datos llamadas *products*. Las colecciones que contenga (el análogo a las tablas en el modelo relacional), las creará la propia aplicación.**




**b. Crear un nuevo usuario. Es recomendable que la contraseña asociada la genere automáticamente la UI de Atlas. Asegúrate de que el nuevo usuario creado dispone de todos los permisos sobre la base de datos.**



## Database Users

| User | Description | Authentication Method | MongoDB Roles | Resources |
|---|---|---|---|---|
| sanasabir94_db_user | | SCRAM | atlasAdmin@admin | All Resources |

c. **<u>Añadir la IP desde la que vas a trabajar</u> a la *whitelist* del clúster (en la UI de Atlas: *Data Services/Security/Network Access*). Puedes saber la IP pública desde la que te estas conectando con esta herramienta, por ejemplo: [https://www.whatismyip.com/es/](https://www.whatismyip.com/es/).**

| IP Address | Comment | Status |
|---|---|---|
| 90.167.228.19/32 (includes your current IP address) | Created as part of the Auto Setup process | Active |

**-Finalmente, añade la URI de la connexión (la UI de Atlas: *Data Services/Database/Connect*) al fichero */config/config.js* del proyecto.**

```
JS config.js ×
M-AC1API > config > JS config.js
1  module.exports = {
2    mongoURI: "mongodb+srv://sanasabir94_db_user:2D
3  };
```

**Tarea 3. Como puedes observar, el repositorio no contiene la carpeta de dependencias *node_modules*. Esta es una buena práctica en tiempo de desarrollo: ¿para qué subir los paquetes de dependencias, que son de largo los más pesados del proyecto, al repositorio (o a Clickedu...), si puede reconstruirse a partir del fichero *package.json*?**

**a) Observa cómo hemos evitado que las dependencias se añadan al control de versiones añadiendo la carpeta *node_modules* al fichero *.gitignore*.**

```
.gitignore
M-AC1API > .gitignore
1   node_modules
2   views/styles/styles.css
```

**b) Observa cómo las dependencias del proyecto están listadas en el fichero *package.json*. Es importante tener este archivo *mantenido*. Si dejamos de utilizar un paquete, hay que *desinstalarlo* para que desaparezca tanto de nuestro directorio de dependencias como del fichero *package.json*.**

```
{} package.json
M-AC1API > {} package.json > ...
1   {
2     "name": "productsapi",
3     "version": "1.0.0",
4     "description": "",
5     "main": "index.js",
      ▷Depurar
6     "scripts": {
7       "test": "echo \"Error: no test specified\" && exit 1"
8     },
9     "author": "",
10    "license": "ISC",
11    "dependencies": {
12      "express": "^4.18.2",
13      "mongodb": "^4.14.0",
14      "pug": "^3.0.2"
15    }
16  }
17
```

**c) Instala las dependencias con *npm install*.**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm install

added 207 packages, and audited 208 packages in 29s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**d) Ejectuta la aplicación (p.ej. *nodemon* app.js) y comprueba que la página de bienvenida de la API se sirve en *http://localhost:5000/api***

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npx nodemon app.js
Need to install the following packages:
nodemon@3.1.14
Ok to proceed? (y) y

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
server is listening on port 5000
```

**Verás que en la página no aplican estilos. Nos ocuparemos de esto más adelante.**

localhost:5000/api

# Wellcome to Products API

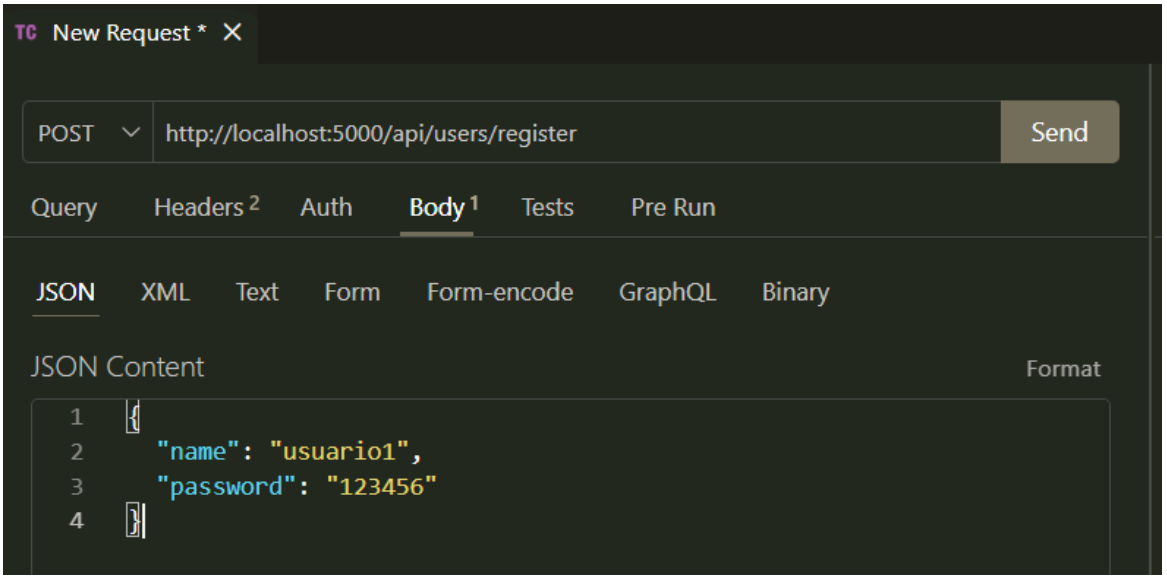
## Register to get API key!

(to provide it, place it on header 'api-key' or as URL param 'apikey')

| Operation | API endpoint |
|---|---|
| Register | POST /api/users/register |
| List all products | GET /api/products |
| List products by ID | GET /api/products/:productID |
| Create product | POST /api/products |
| Update product | PUT /api/products/:productID |
| Delete product | DELETE /api/products/:productID |

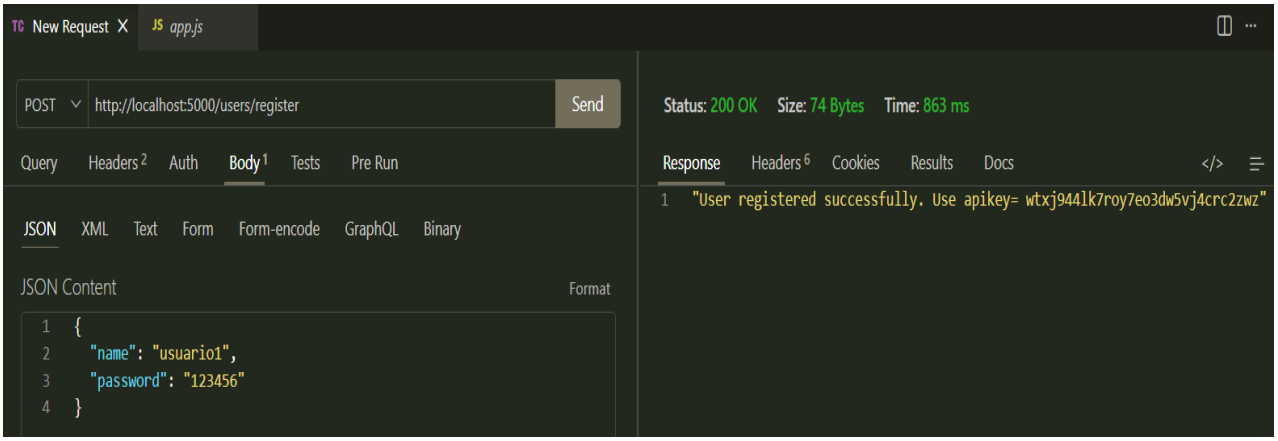
**e) Documenta el correcto funcionamiento de la aplicación:**

**-Registra un nuevo usuario (observa el formato del JSON):**




**Al principio me daba error, pero viendo el archivo app.js , me he cerciorado de que "sobra" lo de "api". Pruebo otra cosa:**

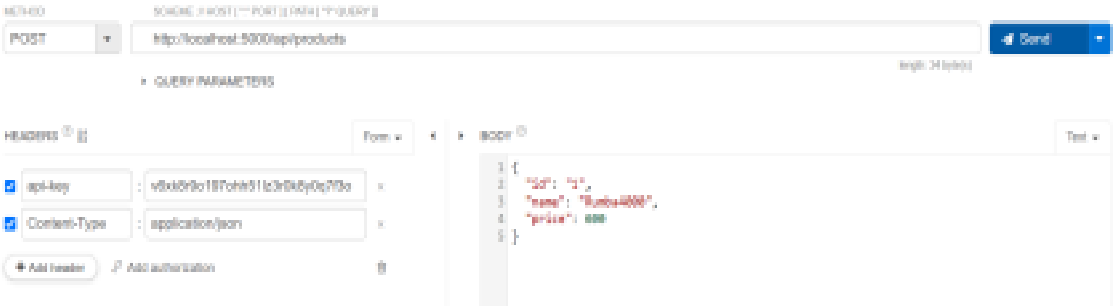
**http://localhost:5000/users/register**

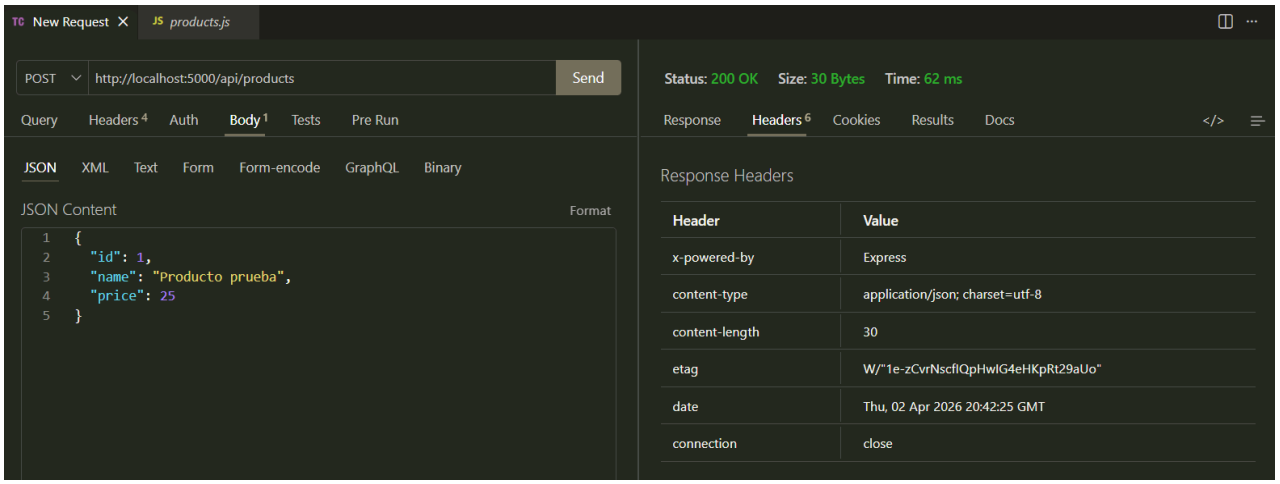


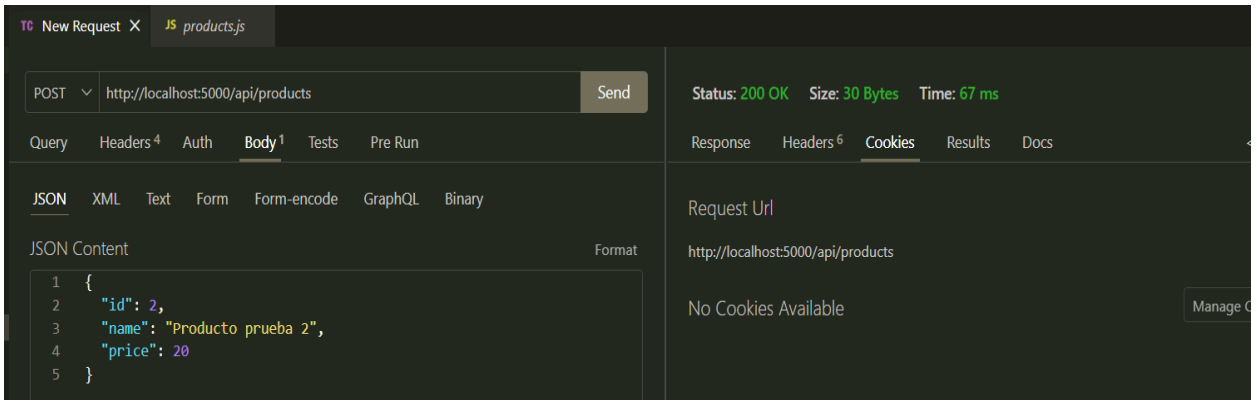
**-Verás que en la respuesta HTTP se proporciona una apikey:**

***"User registered successfully. Use apikey= wtxj944lk7roy7eo3dw5vj4crc2zwz"***

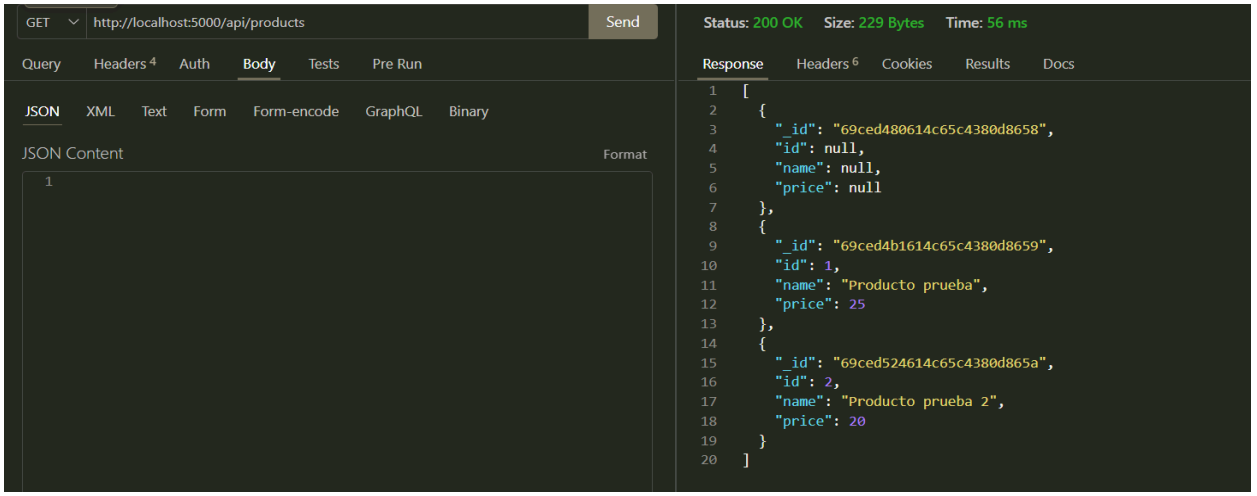
**-Crea algunos productos de prueba:**



**-Consulta la lista de productos**




**f) Finalmente, sube los cambios del proyecto a tu repositorio. Sólo debería haber cambiado el *config.js* (del código fuente). <u>Asegúrate de que no en el *.gitignore* siguen estando la carpeta de dependencias y el fichero CSS.</u>**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git add .

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git commit -m"Config MongoDB y primeras prueba
s"
[main b7d97e4] Config MongoDB y primeras pruebas
 2 files changed, 2840 insertions(+), 1 deletion(-)
 create mode 100644 package-lock.json

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 22.64 KiB | 2.83 MiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ssabir94/M-AC1API.git
   418b820..b7d97e4  main -> main

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**Debo admitir que me he asustado al recibir el correo de MongoDB y el de Github!**

**Tarea 3. Observa que el proyecto usa una VISTA renderizada por el backend, disponible en el endpoint */api/.***

**Investiga en el código del backend:**

**a) Qué motor de plantillas se utiliza.**

**Utiliza "pug".**

**b) Cómo se configura Express para utilizar el motor de vistas.**

***app.set('views', './views');***
***app.set('view engine', 'pug');***

**views → indica dónde están las vistas**
**view engine → indica qué motor se usa (pug)**

## -Despliegue del proyecto online (Render)

**Tarea 4. A continuación, desplegaremos el proyecto *online*. Una buena solución para un aplicación NodeJS sería utilizar el *hosting* de Heroku, pero recientemente han retirado su *free tier*. Por ello, utilizaremos una alternativa parecida:**

**https://render.com/**

**Sigue las instrucciones en este *post* para crear una cuenta en *render.com*, conectarla con tu cuenta de *GitHub* y desplegar el último *commit* realizado.**

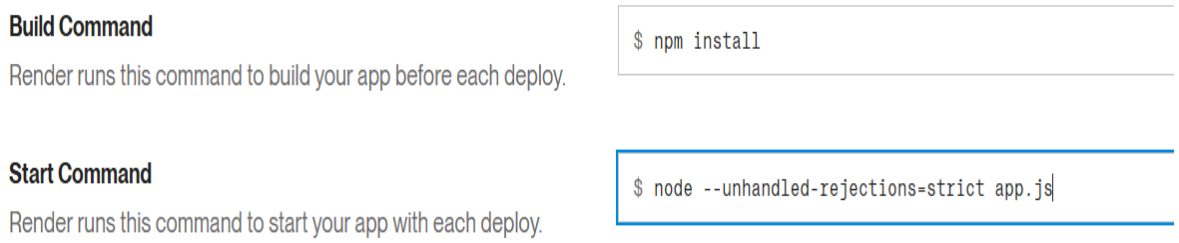
**https://www.freecodecamp.org/news/how-to-deploy-nodejs-application-with**

**-render/ Durante el proceso de instalación, asegúrate de:**

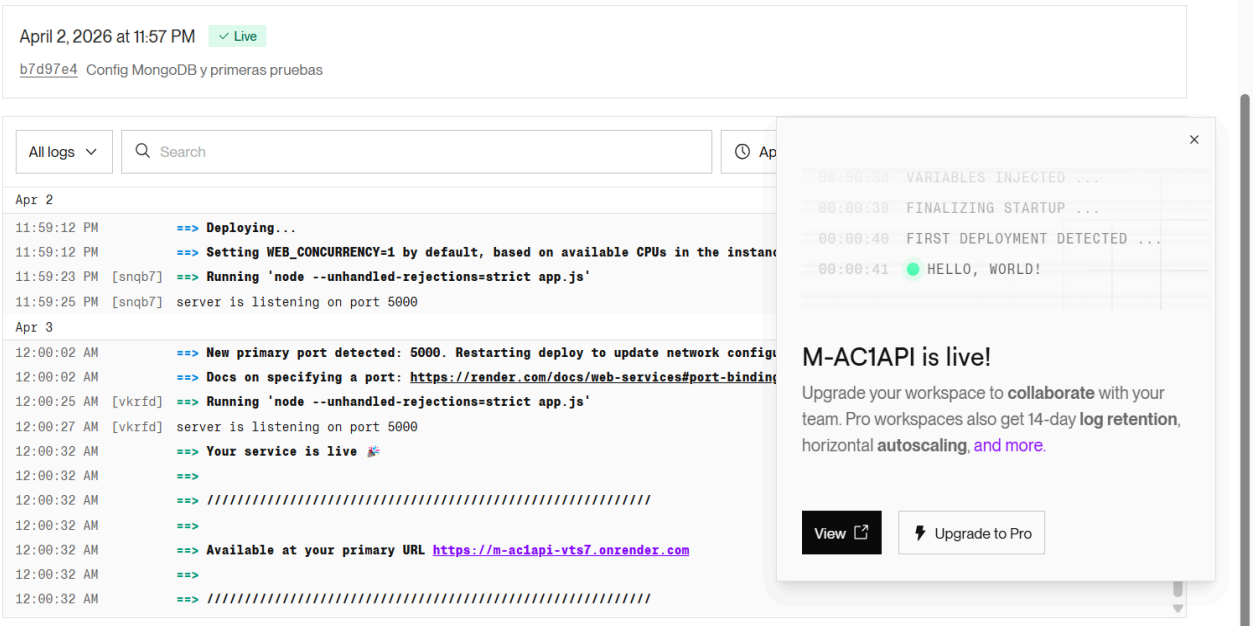
**a) Configurar como comando de compilación (*build command*): *npm install*, ya que las dependencias no se incluyen en el repositorio y deben instalarse en tu sesión del *hosting*.**

**b) Configurar como comando de arranque (*start command*):**
**node --unhandled-rejections=strict app.js**

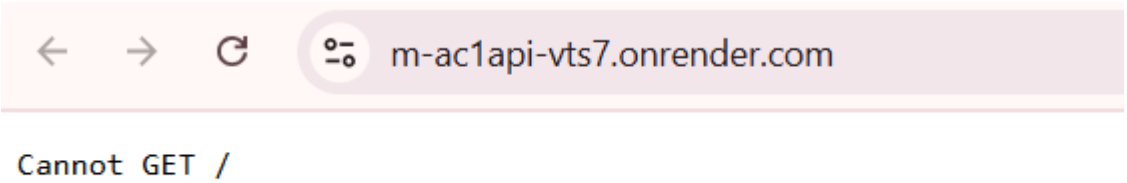
**La opción *unhandled-rejection=strict* soluciona algunos problemas derivados del uso que hacemos del paquete mongodb en relación con la versión de NodeJS que hay instalada en el *hosting*.**



**c) Observa como, después de configurar el nuevo servicio web, se desplega (*deploy*) automáticamente el proyecto del repositorio connectado, en la versión del último commit (<u>si has marcado la opción *autodeploy* durante el proceso de configuración</u>). Captura el *log* del proceso de *deployment*.**




**d) Comprueba el funcionamiento de la api: lo normal sería que pudieras acceder a la página de bienvenida (sin estilos), pero que no pudieras utilizar correctamente el resto de *endpoints*.**




**¿Qué sucede?**

la raíz `/` no está definida

la vista de bienvenida está en `/api`

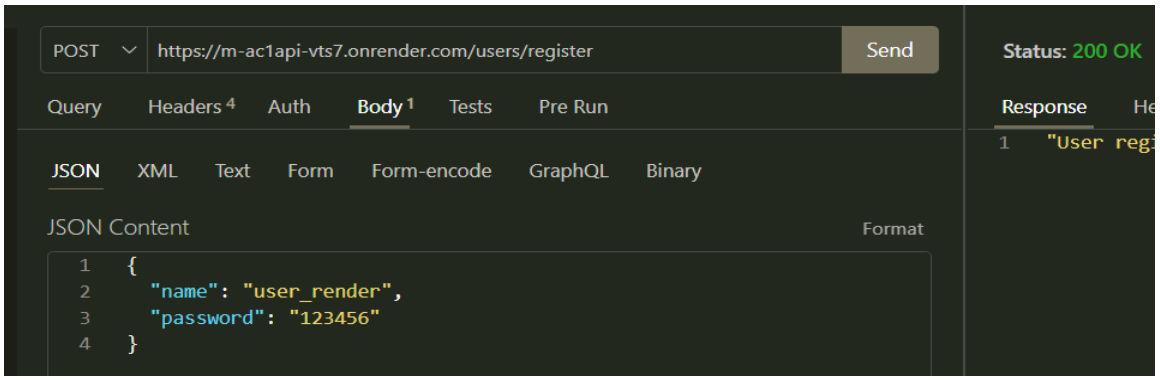
por eso el acceso correcto no es la home del dominio, sino `/api`. Por tanto:

e) Para que éstos funcionen, deberás añadir las IPs del servicio web a la *whitelist* de Atlas. Puedes saber cuáles son las IPs públicas del servicio web en el botón *Connect* del panel de control principal del servicio.

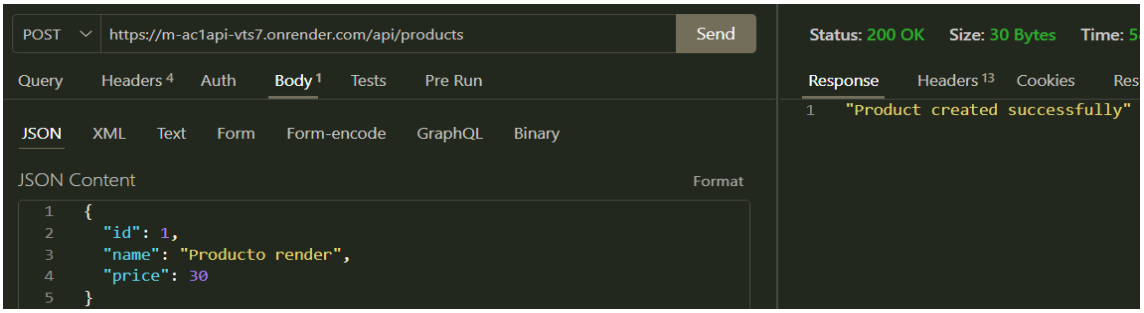
| IP Address | Comment | Status | Actions |
|---|---|---|---|
| 74.220.48.0/24 | | Active | EDIT DELETE |
| 74.220.56.0/24 | | Active | EDIT DELETE |

f) Comprueba el correcto funcionamiento de la API, de manera exhaustiva.

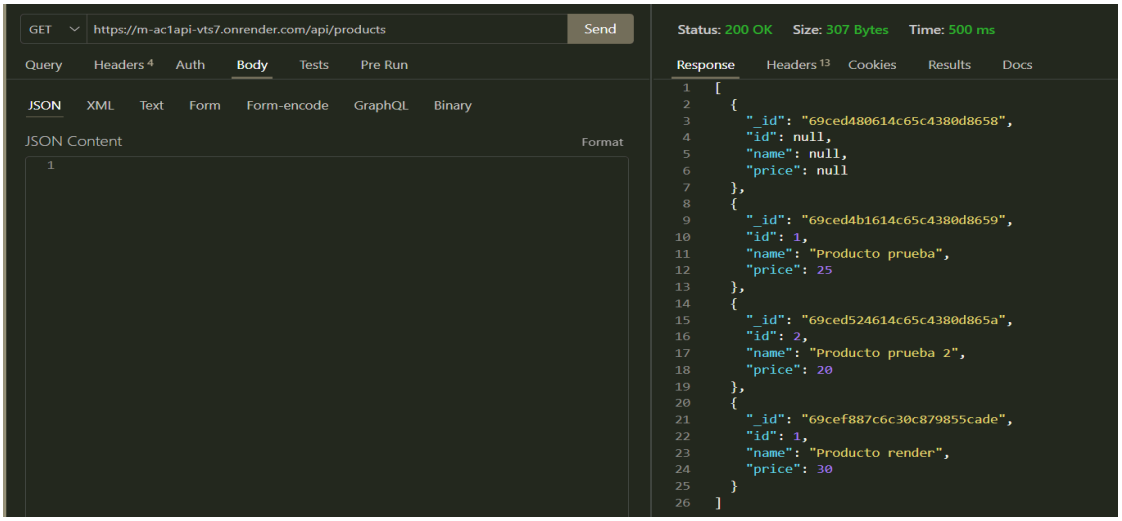
1) *Creamos un registro, y nos dará una nueva ApiKey:*




2) *Creamos producto en Render:*




3) Ahora compruebo:



**Tarea 5. El hecho de que el web service se compile y despliegue automáticamente desde el repositorio de *GitHub* facilita enormemente el proceso de actualización del proyecto. Hagamos un cambio tonto para tener claro el proceso:**

***a)*** **Instala en el proyecto de tu máquina local la dependencia *slugify*. Observa como se ha añadido al *package.json*.**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm install slugify

added 1 package, and audited 209 packages in 6s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

```
  "dependencies": {
    "express": "^4.18.2",
    "mongodb": "^4.14.0",
    "pug": "^3.0.2",
    "slugify": "^1.6.9"
  }
}
```

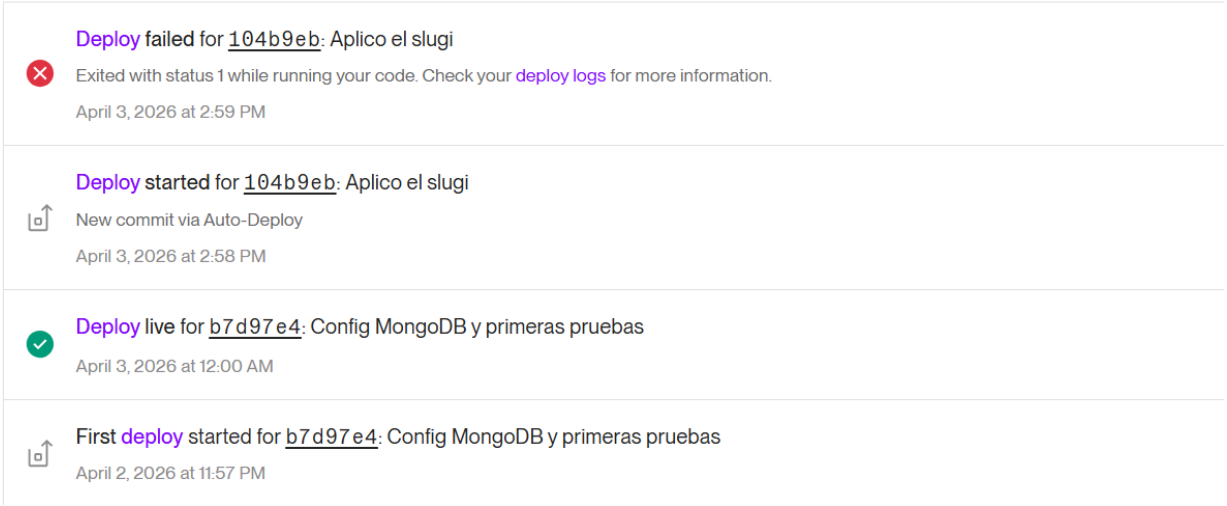
**b) Usa *slugify* para *slugizar* con asteriscos el mensaje de bienvenida del servidor Express (en *app.js*).**

```
//Server startup
app.listen(5000, () => {
    console.log(slugify('server is listening on port 5000', {
  replacement: '*'
}));
})
```

**c) Sube cambios a tu repositorio de *GitHub*.**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git add .

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git commit -m"Aplico el slugi"
[main 104b9eb] Aplico el slugi
 3 files changed, 16 insertions(+), 3 deletions(-)

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 675 bytes | 225.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/ssabir94/M-AC1API.git
   b7d97e4..104b9eb  main -> main
```

**d) En *render,* comprueba cómo se lanza automáticamente el despligue del *commit* que acabas de realizar. Pueder acceder a las trazas del proceso de compilacion y despliegue clicando sobre el *deploy* que toque en el *dashboard* de tu servicio web:**




**Ha ocurrido un error, así que voy a investigar un poco :(**

```
Apr 3
02:59:45 PM  [8b9d9]  Node.js v22.22.0
02:59:48 PM           ==> Exited with status 1
02:59:48 PM           ==> Common ways to troubleshoot your deploy: https://render.com/docs/troubleshooting-deploys
02:59:51 PM  [8b9d9]  ==> Running 'node --unhandled-rejections=strict app.js'
02:59:53 PM  [8b9d9]  /opt/render/project/src/app.js:18
02:59:53 PM  [8b9d9]      console.log(slugify('server is listening on port 5000', {
02:59:53 PM  [8b9d9]              ^
02:59:53 PM  [8b9d9]
02:59:53 PM  [8b9d9]  ReferenceError: slugify is not defined
02:59:53 PM  [8b9d9]      at Server.<anonymous> (/opt/render/project/src/app.js:18:13)
02:59:53 PM  [8b9d9]      at Object.onceWrapper (node:events:633:28)
02:59:53 PM  [8b9d9]      at Server.emit (node:events:531:35)
02:59:53 PM  [8b9d9]      at emitListeningNT (node:net:1983:10)
02:59:53 PM  [8b9d9]      at process.processTicksAndRejections (node:internal/process/task_queues:89:21)
02:59:53 PM  [8b9d9]
02:59:53 PM  [8b9d9]  Node.js v22.22.0
```

**Por lo que parece, el error se debe  a que no hice un const:**

```
M-AC1API > JS app.js > ...
1  const express = require('express')
2  const products_routes = require('./routes/products.js')
3  const slugify = require('slugify');
```

**Ahora, veamos si furula haciendo un push de nuevo:**

**ARA SI!!! 😸**

**e) Accede a la consola del servicio web (sección *logs* del *dashboard*) para comprobar que se han aplicado los cambios correctamente (*slugización* del *prompy* de bienvenida del servidor Express).**

```
02:59:53 PM  [8b9d9]  Node.js v22.22.0
03:00:01 PM  [vpghm]  ==> Running 'node --unhandled-rejections=strict app.js'
03:00:03 PM  [vpghm]  server is listening on port 5000
03:06:43 PM           ==> Deploying...
03:06:43 PM           ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
03:06:55 PM  [hg858]  ==> Running 'node --unhandled-rejections=strict app.js'
03:06:57 PM  [hg858]  server*is*listening*on*port*5000
03:07:04 PM           ==> Your service is live 🎉
03:07:04 PM           ==>
03:07:04 PM           ==> ///////////////////////////////////////////////////////////
03:07:04 PM           ==>
03:07:04 PM           ==> Available at your primary URL https://m-ac1api-vts7.onrender.com
03:07:04 PM           ==>
03:07:04 PM           ==> ///////////////////////////////////////////////////////////
03:12:06 PM           ==> Detected service running on port 5000
03:12:06 PM           ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
```

## -Automatización de la compilación

**Tarea 6. Habitualmente, el proceso de despliegue de una aplicación web habrá de venir precedido no solo de la instalación de las dependencias, sino de la compilación de ciertos ficheros. Estos son algunos ejemplos:**

**-Compilación de SCSS a CSS.**
**-Compilación de *frameworks* JS: REACT, Vue...**
**-Transpilación de Typescript a Javascript (en Angular, por ejemplo).**
**-Post-procesado de CSS (autoprefixing, por ejemplo).**
**-Minificación de ficheros.**
**-Linting de código.**

**En esta ocasión, necesitamos automatizar el proceso de compilación de CSS a partir de ficheros SASS. Para ello:**

**a) Instala el paquete *node-sass* en tu proyecto local, como <u>depedencia de</u> <u>desarrollo</u>: *npm install --save-dev node-sass*. Observa como se añade al *package.json.***

**Parece que está obsoleto, y me ha dado varios errores. Procedo a desinstalar:**

```
npm error gyp ERR! cwd C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API\node_modules\node-sass
npm error gyp ERR! node -v v22.14.0
npm error gyp ERR! node-gyp -v v8.4.1
npm error gyp ERR! not ok
npm error Build failed with error code: 1
npm error A complete log of this run can be found in: C:\Users\HP\AppData\Local\npm-cache\_logs\2026-04-03T13_26_35_676Z
-debug-0.log

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm uninstall node-sass

up to date, audited 209 packages in 2s

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**Voy a intentarlo de otra manera:**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm install --save-dev sass

added 12 packages, and audited 221 packages in 4s

32 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**b) Lanza la compilación de estilos mediante *npx:***

***npx node-sass ./views/styles/main.scss ./views/styles/styles.css***

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npx sass ./views/styles/main.scss ./views/styl
es/styles.css

C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>
```

**c) Observa cómo se genera el fichero de estilos. Ahora la página de bienvenida de la API aparecerá correctamente estilada.**

## He tenido que hacer cambios en pug, porque sino no aparecia:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>API How To</title>
    <link rel="stylesheet" href="styles/styles.css">
</head>
<body>
    h2=titulo
    h3=aviso
    p=howto
    table
        th="Operation"
        th="API endpoint"
        each ruta in rutas
            tr
                each val in ruta
                    td= val
</body>
</html>
```

localhost:5000/api

**Wellcome to Products API**

**Register to get API key!**

(to provide it, place it on header 'api-key' or as URL param 'apikey')

| Operation | API endpoint |
|---|---|
| Register | POST /api/users/register |
| List all products | GET /api/products |
| List products by ID | GET /api/products/:productID |
| Create product | POST /api/products |
| Update product | PUT /api/products/:productID |
| Delete product | DELETE /api/products/:productID |

**d) Convertiremos ahora el comando de compilación de estilos en un *script npm*. Para ello, añade una nueva clave en el apartado de *scripts* del *package.json*:**

**"build:css": "node-sass ./views/styles/main.scss ./views/styles/styles.css"**

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "build:css": "sass ./views/styles/main.scss ./views/styles/styles.css",
```

**e) Haz algún cambio en el SASS y recompila lo estilos, esta vez usando el script npm: npm run build:css.**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm run build:css

> productsapi@1.0.0 build:css
> sass ./views/styles/main.scss ./views/styles/styles.css
```

**f) Finalmente, empaquetaremos las dos operaciones (instalación de depencias y compilación de estilos) en un único *script npm*:**

**"build": "npm install && npm run build:css",**

```
  "build": "npm install && npm run build:css"
},
```

**g) Actualiza el compando de compilación (*build command*) en los *settings* del servicio web en render a: *npm run build*.**

**Build Command**

Render runs this command to build your app before each deploy.

```
$ npm run build
```

**Y… falló**

```
Apr 3
04:37:00 PM ==> Downloading cache...
04:37:00 PM ==> Cloning from https://github.com/ssabir94/M-AC1API
04:37:01 PM ==> Checking out commit beb65a8445bbc74830298b420c00fe8b33d60ddd in branch main
04:37:08 PM ==> Downloaded 65MB in 4s. Extraction took 2s.
04:37:48 PM ==> Using Node.js version 22.22.0 (default)
04:37:48 PM ==> Docs on specifying a Node.js version: https://render.com/docs/node-version
04:37:48 PM ==> Running build command 'npm run build'...
04:37:48 PM npm error Missing script: "build"
04:37:48 PM npm error
04:37:48 PM npm error To see a list of scripts, run:
04:37:48 PM npm error   npm run
04:37:48 PM npm error A complete log of this run can be found in: /opt/render/.cache/_logs/2026-04-03T14_37_48_465Z-debug-0.log
04:37:48 PM ==> Build failed 😞
04:37:48 PM ==> Common ways to troubleshoot your deploy: https://render.com/docs/troubleshooting-deploys
```

**Vamos a investigar. Por lo que parece, se debe a que no he hecho los cambios en Github, y por ende, no se aplica a Render.**

**h) Sube los cambios al repo, observa cómo se lanza un nuevo despliegue en render, y como, ahora sí, se genera el fichero de estilos (que, recordemos, NO SE SUBE AL REPO), y la página de bienvenida de la API *online* aparece estilada correctamente.**

**En el siguiente enlace puedes ver otros *scripts npm* útiles para la automatización de procesos de despliegue:**
**[https://css-tricks.com/why-npm-scripts/](https://css-tricks.com/why-npm-scripts/)**

Deploy live for `6f5eb7d`: Cambios CsS y Render
April 3, 2026 at 4:56 PM

Deploy started for `6f5eb7d`: Cambios CsS y Render
New commit via Auto-Deploy
April 3, 2026 at 4:55 PM

```
Apr 3
04:55:42 PM          > sass ./views/styles/main.scss ./views/styles/styles.css
04:55:42 PM
04:55:44 PM          ==> Uploading build...
04:55:50 PM          ==> Uploaded in 4.2s. Compression took 1.7s
04:55:50 PM          ==> Build successful 🎉
04:55:53 PM          ==> Deploying...
04:55:53 PM          ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
04:56:06 PM  [p7nh2] ==> Running 'node --unhandled-rejections=strict app.js'
04:56:08 PM  [p7nh2] server*is*listening*on*port*5000
04:56:14 PM          ==> Your service is live 🎉
04:56:15 PM          ==>
04:56:15 PM          ==> ///////////////////////////////////////////////////////////
04:56:15 PM          ==>
04:56:15 PM          ==> Available at your primary URL https://m-ac1api-vts7.onrender.com
04:56:15 PM          ==>
04:56:15 PM          ==> ///////////////////////////////////////////////////////////
```

localhost:5000/api

**Wellcome to Products API**

**Register to get API key!**

(to provide it, place it on header 'api-key' or as URL param 'apikey')

| Operation | API endpoint |
|---|---|
| Register | POST /api/users/register |

## -Ocultación de secretos:

**Tarea 7. Lo más probable es que *GitHub* te haya avisado, mediante un correo, de que existe en tu repositorio una vulnerabilidad de seguridad. Concretamente, de que la URI de la conexión a la base de datos de MongoDB está expuesta, en texto plano, en el repositorio. Esto es gravísimo.**

**a) Vamos a convertir la URI a una variable de entorno para que no quede almacenada en el código. Para ello, desde la terminal integrada de VSCode, usa:**

**$ENV:MONGOURI='tuURI'**

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>set MONGOURI=mongodb+srv://
```

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "build:css": "sass ./views/styles/main.scss ./views/styles/styles.css",
  "build": "npm install && npm run build:css",
  "start": "node app.js"
```

***He añadido eso para que funcione "npm start"***

**b) Ahora, haz que en el fichero *config.js* se lea esta URI desde la variable de entorno: process.env. MONGOURI;**

```
M-AC1API > config > JS config.js > ...
1  module.exports = {
2    mongoURI: process.env.MONGOURI
3  };
```

**c) Comprueba que la API sigue pudiendo conectar correctamente a la base de datos.**

localhost:5000/api

**Wellcome to Products API**

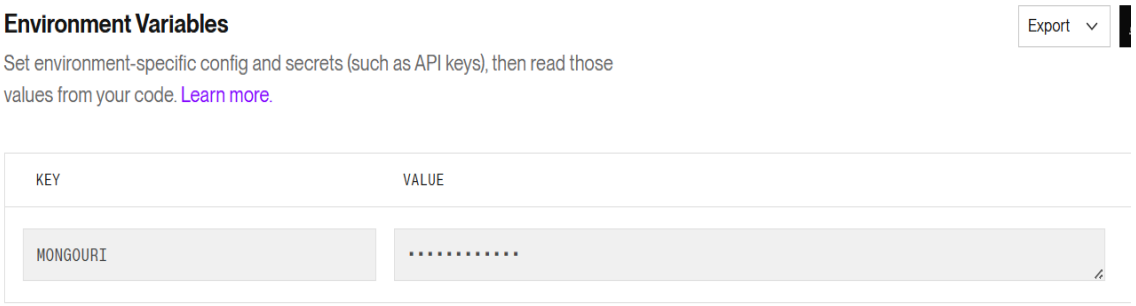
**Register to get API key!**

(to provide it, place it on header 'api-key' or as URL param 'apikey')

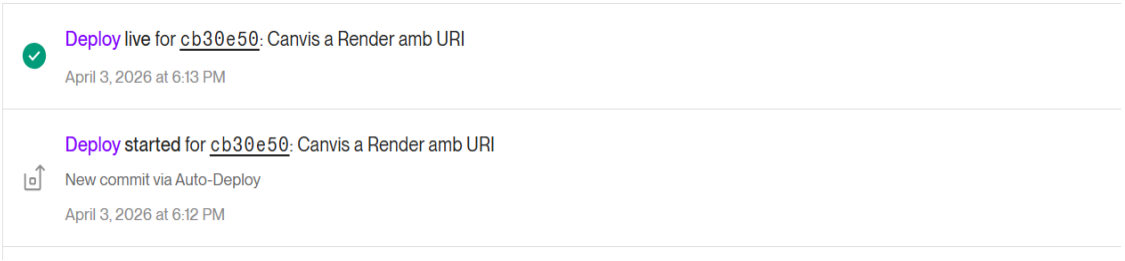
| Operation | API endpoint |
|---|---|
| Register | POST /api/users/register |
| List all products | GET /api/products |

```
C:\Users\HP\Documents\DAW 2N\DESPLEGAMENT D'APLICACIONS WEB\AC31\M-AC1API>npm start

> productsapi@1.0.0 start
> node app.js

server*is*listening*on*port*5000
Connected successfully to database (MongoDB)
User not authenticated
```

**d) En el panel de control del servicio web en render, en la sección *Environment*, añade la variable de entorno a la sesión del *hosting*.**




**e) Sube los cambios locales al repositorio, observa como se lanza otro *deploy*, y comprueba que la API *online* sigue funcionando. La URI de MongoDB ha dejado de estar expuesta en el repositorio.**




```
06:13:19 PM          > sass ./views/styles/main.scss ./views/styles/styles.css
06:13:19 PM
06:13:21 PM          ==> Uploading build...
06:13:26 PM          ==> Uploaded in 3.4s. Compression took 1.6s
06:13:26 PM          ==> Build successful 🎉
06:13:32 PM          ==> Deploying...
06:13:32 PM          ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
06:13:45 PM  [tld68] ==> Running 'node --unhandled-rejections=strict app.js'
06:13:47 PM  [tld68] server*is*listening*on*port*5000
06:13:54 PM          ==> Your service is live 🎉
06:13:54 PM          ==>
06:13:54 PM          ==> ///////////////////////////////////////////////////////////
06:13:54 PM          ==>
06:13:54 PM          ==> Available at your primary URL https://m-ac1api-vts7.onrender.com
06:13:54 PM          ==>
06:13:54 PM          ==> ///////////////////////////////////////////////////////////
```

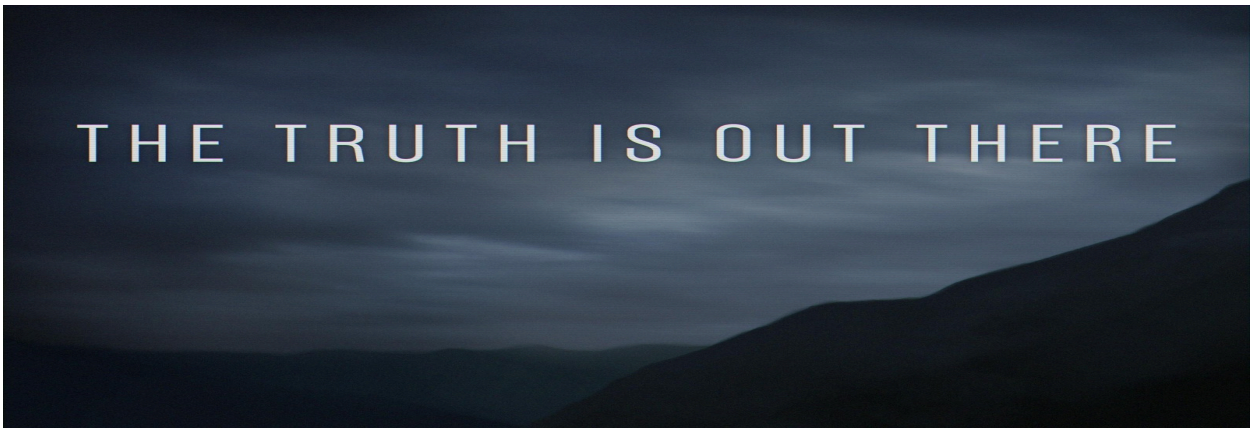
**Esta manera de trabajar con variables de entorno no es la óptima. Existen algunos paquetes (p.ej. *dotenv*) que automatizan la configuración en el entorno de estas variables a partir de ficheros dedicados de extensión *.env*, que OBVIAMENTE, deben añadirse al *.gitignore*, para no quedar expuestos en el repo, y subirse a mano al *hosting*. Investiga en qué lugar se suben este tipo de ficheros en el caso de render.**

**Aunque existen paquetes como dotenv para leer variables desde un fichero .env en local, en Render estos valores no se suben al repositorio. En su lugar, se añaden desde la sección Environment del servicio. Render permite importarlos en bloque con la opción "Add from .env".**

**Además, si se requiere subir un archivo secreto como tal, se puede hacer desde Environment > Secret Files.**

## CONCLUSIÓN:

**Ha sido un trabajo con el que he sudado un poco (básicamente en el momento en el que GitHub y MongoDB me decían que tenía datos vulnerables al descubierto xD ), y he tenido que dedicar bastantes horas.**

**Me gusta la idea de documentarlo todo, para poder usarlo en un futuro.**

***Y para terminar, haciendo referencia a los Secret Files… Como diría Mulder:***